UNIT 4

Testing

Testing is the activity where the errors remaining from all the previous phases must be detected. Testing performs a very critical role for quality assurance and for ensuring the reliability of software.

During testing, the software to be tested is executed with a set of test cases and behavior of the system for the test case is evaluated to determine if the system is performing as expected.

Incremental testing is performed for the large and complex software system, in which components and sub systems are tested separately before integrating them to form the system for system testing. This form of testing sometimes referred as "integration and testing".

There are two basic approaches to testing –

I. *Black box testing* or functional testing
II. *White box* testing or structural testing

Error, Fault and Failure

The term Error is used in two different ways,

i) It refers to the difference between the actual output of the software and the correct output. Here the error is measure of the actual and ideal.

ii) It refers to human actions that result in software containing a defect or fault.

Fault is a condition that ensures a system to fail in performing its required function. It is the basic reason for software malfunction and a synonymous with the commonly known as bug.

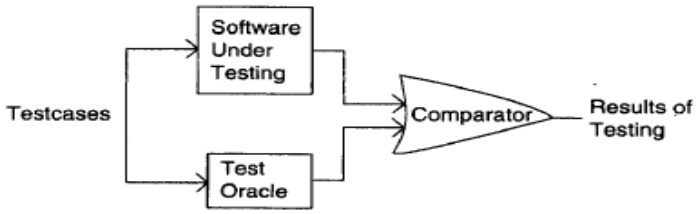
Failure is the inability of a system or component to perform a required function according to its specifications. A software failure occurs if the behavior of the software is different from the specified behavior. Failures may be caused due to functional or performance reasons. A failure is produced only when there is a fault in the system.

For a system,

- Faults have the potential to cause failures.
- Presence of an error implies that a failure must have occurred and observance of a failure implies that a fault must be present in the system.
- Testing only reveals the presence of failure; the actual faults are identified by debugging.

Test Oracles

A test oracle is a mechanism, different from the program itself that can be used to check the correctness of the output of the program for the test cases.



Testing and Test Oracles

Here, we can consider testing a process in which the test cases are given to the test oracle and the program under testing. The output of the two is then compared to determine if the program behaved correctly for the test cases.

Test oracles are necessary for testing. Ideally, we would like an automated oracle, which always gives a correct answer. However, often the oracles are human beings, who can make mistakes. As a result, when there is a discrepancy between the results of the program and the oracle, we have to verify the result produced by the oracle, before declaring that there is a fault in the program.

The human oracles generally use the specifications of the program to decide what the "correct" behavior of the program should be.

An oracle generated from the specifications will only produce correct results if the specifications are correct.

Top-Down and Bottom-Up approaches

For a large system, we should first test different parts of the system independently before testing the entire system. In incremental testing. Some parts of the system are first tested independently then these parts are combined to form a subsystem which is then test independently.

In **top-down strategy,** we start by testing the top of the hierarchy and incrementally add modules that it calls and then test the new combined system. This approach of testing requires stubs to be written. To allow the modules to be tested before their subordinates have been coded, stubs simulate the behavior of the subordinates.

The **bottom-up approach** starts from bottom of the hierarchy. First the modules at the very bottom, which have no subordinates, are tested. Then these modules are combined with higher

level modules for testing. To perform bottom-up testing drivers are need to set up the appropriate environment and invoke the module.

A stub is dummy routine that simulate a module.
Top-down testing is advantageous if major flows occur towards the top of the hierarchy.
Bottom-up testing is advantageous if the major flows occur toward the bottom.

Test Cases and Test Criteria

If testing has to be successful, then we should have test cases that are good at revealing the presence of faults. We would like to determine a set of test cases such that successful.
Normally in testing we have to determine a set of test cases such that successful execution of all of them implies that there are no errors in the program. Generally we have to minimize the test cases costly and required more effort.
The fundamental goals of a practical testing are

- Maximize the number of errors detected.
- Minimize the number of test cases.

An ideal test case set is one that succeeds only if there are no errors in the program. One possible ideal test case is one that includes all the possible inputs to the program. This is often called exhaustive testing.
Test Selection Criterion (test criterion) is used for selecting test cases. For a given program P and its specification S, a test selection criterion specified by a set of a test case T. the criterion becomes a basis for test case selection.
There are two fundamental properties for a testing criterion: reliability and validity.
A criterion is reliable if all the sets (of test cases) that satisfy the criterion detect the same errors.
A criterion is valid if for any error in the program there is some set satisfying the criterion that will reveal the error.
Getting an ideal test criterion is not generally possible so that more practical properties of test criteria have been proposed. Some axioms capturing some of the desirable properties of test criteria have been proposed.
Applicability Axiom: it states that for every program there exists a test case set T that satisfies the criterion.

Antiextentionality Axiom: it states that there are programs P and Q, both implementing the some specifications, such that a test set T satisfy the criterion for P but does not satisfy the criterion Q.
Antidecomposition Axiom: it states that there exists a program P and its component Q such that a test case set T satisfies the criterion for P and T ' is the set of values that variables can assume on entering Q for some test case T and T ' does not satisfy the criterion Q.
Anticomposition Axiom: it states that there exists programs P and Q such that T satisfies the criterion for P and the outputs of P for T satisfies the criterion Q, but T does not satisfy the criterion for P;Q.
Getting a criterion that is reliable and valid and that can be satisfied by a manageable number of test cases is usually not possible. Even when the criterion is specified, generating test cases to satisfy a criterion is not simple. In general, generating test cases for most of the criteria cannot be automated.

Comparison of criteria

A criterion C1 includes the criterion C2 if for every program P and its specification S, any set of test cases that satisfy C1 also satisfy C2. This relation is represented as C1 ➔ C2, and is a transitive relation.
One may think that if C1 ➔ C2, testing based on C1 will always be better than testing based on C2. Unfortunately, this is not the case. The reason is that the fault-detection capability of a set of test cases T that satisfy a criterion C depends on the actual test cases in T and not just C (i.e., the criterion is not valid).

Psychology of Testing

A set of test cases that will guarantee that all errors will be detected is not feasible. Moreover, there are no formal or precise methods for selecting test cases. Even though there are a number of heuristics and rules of thumb for deciding the test cases, selecting test cases is still a creative activity that relies on the ingenuity of the tester. Because of this, the psychology of the person performing the testing becomes important.
The basic purpose of testing is to detect the errors that may be present in the program. Hence, one should not start testing with the intent of showing that a program works; but the intent should be to show that a program does not work. With this in mind we can define testing as the process of executing a program with the intent of finding errors.

Functional Testing (black-box testing)

There are 2 approaches to testing functional (black-box) and structural (white-box).

In **functional testing** the structure of the program is not considered. Test cases are decided on the basis of requirements of specifications of the program or module and internal of the modules or the programs are not considered for selection of test cases. Due to its nature functional testing also called as **black-box** testing.

In **structural testing** test cases are generated based on actual code of the program or module to be tested, this structural approach sometimes are called as **white-box testing** or **glass-box testing**.

The basis for deciding test cases in functional testing is requirements or specifications of the system or module. For modules created during design, test cases for functional testing are decided from the module specifications produced during the design.

There are no formal rules for designing test cases for functional testing. In fact there are no precise criteria for selecting test cases. There are a number of techniques and heuristics that can be used to select test cases that have been found to be very successful in detecting errors.

Equivalence Class Partitioning

The natural approach is to divide the input domain into a set of equivalence classes, so that if the program works correctly for a value then it will work correctly for all the other values in that class. If we can indeed identify such classes, then testing the program with one value from each equivalence class is equivalent to doing an exhaustive test of the program.

The equivalence class partitioning method tries to approximate this ideal. An equivalence class is formed of the inputs for which the behavior of the system is specified or expected to be similar. Each group of inputs for which the behavior is expected to be different from others is considered a separate equivalence class.

For example, the specifications of a module that determines the absolute value for integers specify one behavior for positive integers and another for negative integers. In this case, we will form two equivalence classes—one consisting of positive integers and the other consisting of negative integers. Equivalence classes are usually formed by considering each condition specified on an input as specifying a valid equivalence class and one or more invalid equivalence classes.

For example, if an input condition specifies a range of values (say, 0 < count < Max), then form a valid equivalence class with that range and two invalid equivalence classes, one with values less than the lower bound of the range (i.e., count < 0) and the other with values higher than the higher bound (count > Max). If the input specifies a set of values and the requirements specify different behavior for different elements in the set, then a valid equivalence class is formed for each of the elements in the set and an invalid class for an entity not belonging to the set.

For an output equivalence class, the goal is to have inputs such that the output for that test case lies in the output equivalence class. Determining test cases for output classes may be more difficult, but output classes have been found to reveal errors that are not revealed by just considering the input classes.

As an example consider a program that takes two inputs—a string **s** of length up to **N** and an integer **n**. The program is to determine the top n highest occurring characters in s. The tester beUeves that the programmer may deal with different types of characters separately. One set of valid and invalid equivalence classes for this is shown in Table

Input	**Valid Equivalence Classes**	**Invalid Equivalence Classes**
S	EQ1: Contains numbers EQ2: Contains lower case letters EQ3: Contains upper case letters EQ4: Contains special characters EQ5: String length between 0-N	lEQ1: non-ASCII characters IEQ2: String length > N
N	EQ6: Integer in valid range	IEQ3: Integer out of range

With these as the equivalence classes, we have to select the test cases. A test case for this is a pair of values for s and n. With the first strategy for deciding test cases, one test case could be: 5

as a string of length less than N containing lower case, upper case, numbers, and special characters; and n as the number 5. This one test case covers all the valid equivalence classes (EQ1 through EQ6). Then we will have one test case each for covering lEQ1, IEQ2, and IEQ3. That is, a total of 4 test cases is needed.

Boundary Value Analysis

It has been observed that programs that work correctly for a set of values in an equivalence class fail on some special values. These values often lie on the boundary of the equivalence class. Test cases that have values on the boundaries of equivalence classes are therefore likely to be "high-yield" test cases, and selecting such test cases is the aim of the boundary value analysis.

In **boundary value analysis**, we choose an input for a test case from an equivalence class, such that the input lies at the edge of the equivalence classes. Boundary values for each equivalence class, including the equivalence classes of the output, should be covered. Boundary value test cases are also called "extreme cases." Hence, we can say that a boundary value test case is a set of input data that lies on the edge or boundary of a class of input data or that generates output that lies at the boundary of a class of output data.

In case of ranges, for boundary value analysis it is useful to select the boundary elements of the range and an invalid value just beyond the two ends (for the two invalid equivalence classes). So, if the range is $0.0 < x < 1.0$, then the test cases are 0.0, 1.0 (valid inputs), and -0.1, and 1.1 (for invalid inputs).

Similarly, if the input is a list, attention should be focused on the first and last elements of the list. We should also consider the outputs for boundary value analysis. If an equivalence class can be identified in the output, we should try to generate test cases that will produce the output that lies at the boundaries of the equivalence classes. Furthermore, we should try to form test cases that will produce an output that does not lie in the equivalence class.

Cause –Effect graphing

Cause-effect graphing is a technique that aids in selecting combinations of input conditions in a systematic way, such that the number of test cases does not become unmanageably large. The technique starts with identifying causes and effects of the system under testing. A cause is a

distinct input condition, and an effect is a distinct output condition. Each condition forms a node in the

cause-effect graph. The conditions should be stated such that they can be set to either true or false.

For example, an input condition can be "file is empty," which can be set to true by having an empty input file, and false by a nonempty file. After identifying the causes and effects, for each effect we identify the causes that can produce that effect and how the conditions have to be combined to make the effect true.

Conditions are combined using the Boolean operators "AND," "OR," and "NOT," which are represented in the graph by &, |, and ~. Then for each effect, all combinations of the causes that the effect depends on which will make the effect true are generated (the causes that the effect does not depend on are essentially "don't care"). By doing this, we identify the combinations of conditions that make different effects true. A test case is then generated for each combination of conditions, which make some effect true.

Let us illustrate this technique with a small example. Suppose that for a bank database there are two commands allowed:

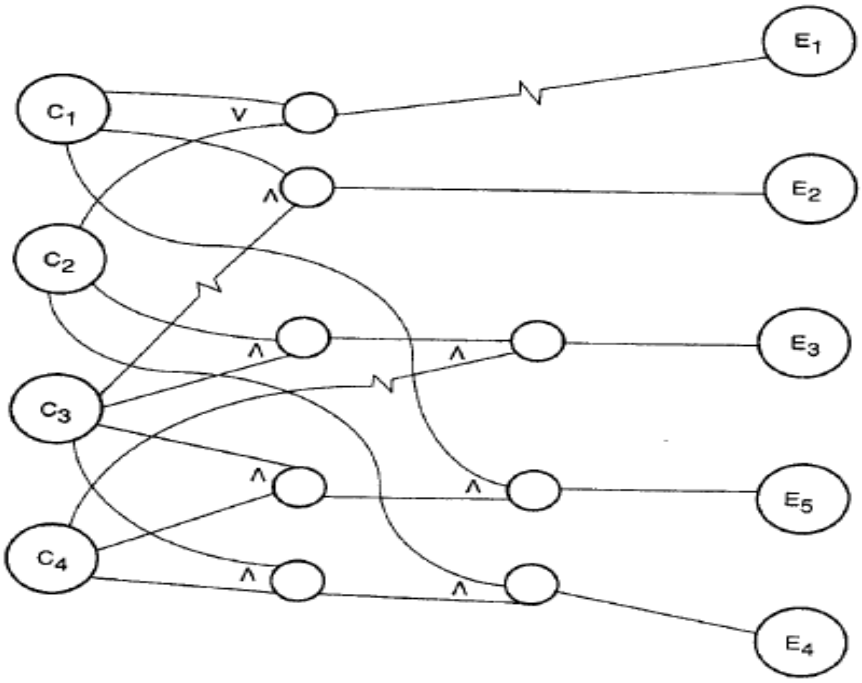
credit acct_number transaction_amount

debit acct_number transaction_amount

The requirements are that if the command is credit and the acct_number is valid, then the account is credited. If the command is debit, the acct.number is valid, and the transaction_amount is valid (less than the balance), then the account is debited. If the command is not valid, the account number is not valid, or the debit amount is not valid, a suitable message is generated. We can identify the following causes and effects from these requirements:

Causes:	Effects:
cl. Command is credit	el. Print "invahd command"
c2. Command is debit	e2. Print "invalid account_number"
c3. Account number is valid	e3. Print "Debit amount not vahd"
c4. Transaction_amt is valid	e4. Debit account
	e5. Credit account

the cause-effect graph is as shown below



cause-effect graph

In the above graph, the cause-effect relationship of this example is captured. For all effects, one can easily determine the causes each effect depends on and the exact nature of the dependency. For example, according to this graph the effect e5 depends on the causes c2, c3, and c4 in a manner such that the effect e5 is enabled when all c2, c3, and c4 are true. Similarly, the effect e2 is enabled if c3 is false.

From this graph, a Hst of test cases can be generated. The basic strategy is to set an effect to 1 and then set the causes that enable this condition. The condition of causes forms the test case. A cause may be set to false, true, or don't care (in the case when the effect does not depend at all on the cause). To do this for all the effects, it is convenient to use a decision table.

The *decision table* for this example is shown bellow.

SNo.	1	2	3	4	5
c1	0	1	x	x	1
C2	0	x	1	1	x
c3	x	0	1	1	1
c4	x	x	0	1	1
e1	1				
e2		1			
e3			1		
e4				1	
e5					1

Table: Decision table for cause-effect graph

This table lists the combinations of conditions to set different effects. Each combination of conditions in the table for an effect is a test case.

Together, these condition combinations check for various effects the software should display. For example, to test for the effect e3, both c2 and c4 have to be set. That is, to test the effect "Print debit amount not valid," the test case should be: Command is debit (setting c2 to True), the account number is valid (setting c3 to False), and the transaction money is not proper (setting c4 to False).

Cause-effect graphing, beyond generating high-yield test cases, also aids the understanding of the functionahty of the system, because the tester must identify the distinct causes and effects. There are methods of reducing the number of test cases generated by proper traversing of the graph. Once the causes and effects are listed and their dependencies specified, much of the remaining work can also be automated.

Structural testing/whitebox testing.

Structural testing, on the other hand is concerned with testing the implementation of the program. The intent of this testing is not to exercise all the different input or output conditions but to exercise the different programming structures and data structures used in the program. structural testing is also called white-box testing.

To test the structure of a program, structural testing aims to achieve test cases that will force the desired coverage of different structures. The criteria for structural testing are generally quite

precise as they are based on program structures, which are formal and precise. There are three basic structural testing approaches

1. Control flow based testing
2. Data flow based testing
3. Mutation testing.

Control flow based testing